Why CP-SAT for OptiFlux Crew Recovery

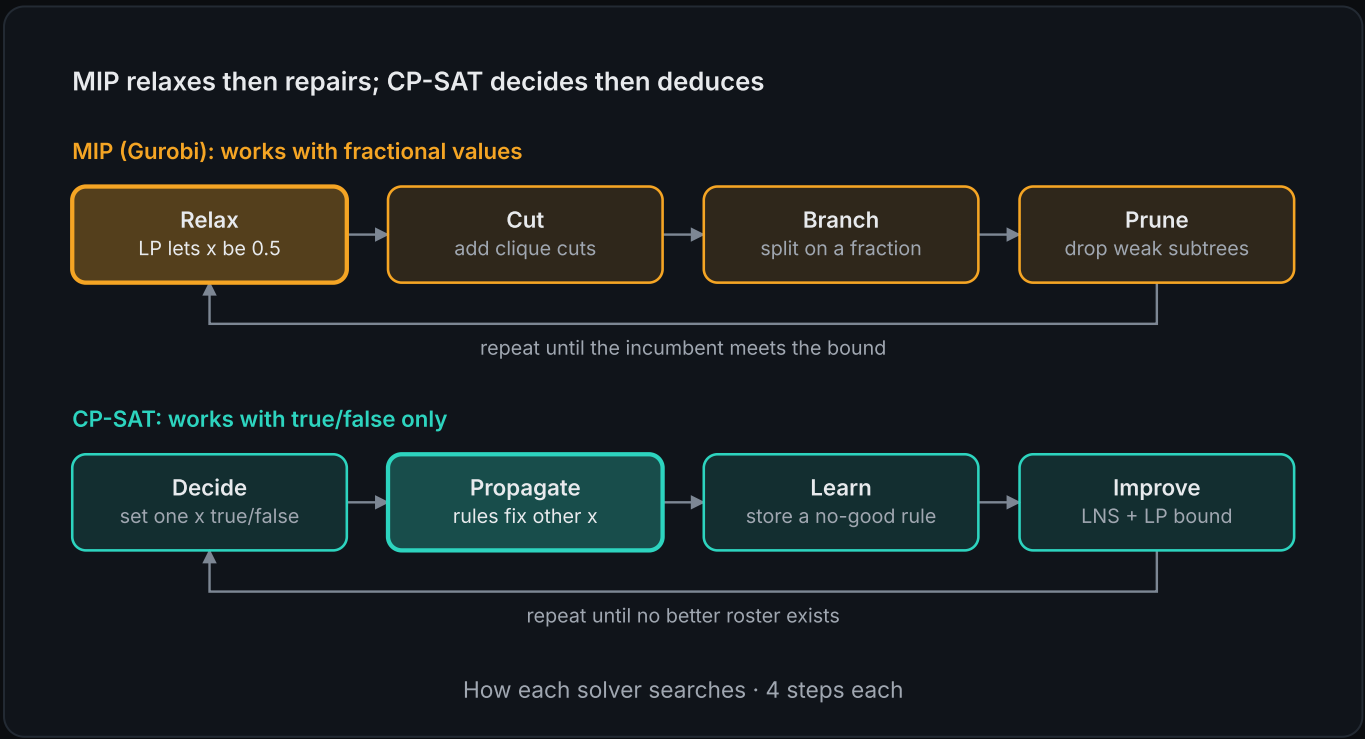
Oct 27, 2026 · By Sourendra

Summary

We recommend Google OR-Tools CP-SAT as the primary solver for OptiFlux crew recovery: about 300 uncovered 12-hour duties, about 1,000 crew, a 14-day horizon, and swing, shift-transition, CO and return-to-base rules. Four reasons:

1. **It remembers dead ends.** When a combination of assignments fails, CP-SAT stores the cause as a new rule and never tries that combination again anywhere in the search. Branch and bound in MIP prunes mainly by LP bounds, so it can walk into the same dead end in many branches.
2. **Our rules are logical, not just linear.** Swing patterns, transition-day rest, lift-up/layback, CO restrictions and return to base are if-then rules. CP-SAT expresses them directly; MIP needs big-M constraints that weaken its bounds.
3. **It finds good rosters fast and says how good they are.** It returns a feasible roster early, improves it with parallel local search, and reports an upper bound on coverage.
4. **It is free, Python-native and strong.** It runs in Colab with one `pip install` and generally outperforms the free MIP solvers on combinatorial problems like ours.

MIP is not wrong for this domain. At full industrial scale a column-generation MIP is a likely end state, as covered in "Where MIP still wins".

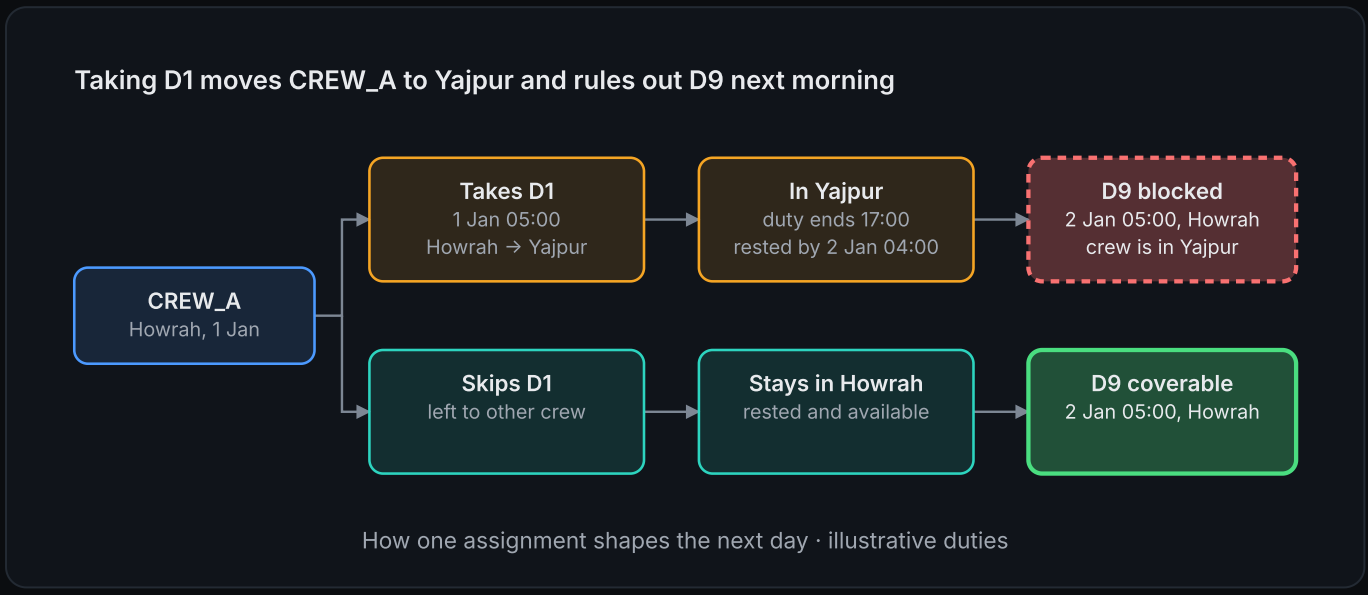


The highlighted step is each solver's core engine: the LP relaxation for MIP, propagation for CP-SAT.

The problem we are solving

We assign available crew to about 300 uncovered 12-hour duties over a horizon of up to 14 days, with about 1,000 crew and up to 50 eligible crew per duty. These are scoping estimates, not validated dataset statistics. Routes are Howrah ↔ Yajpur and Howrah ↔ Nellore.

The hard part is **sequencing**. A duty can be feasible on its own and still make later duties impossible, because each duty moves the crew member to a new location and starts a rest clock.



How one assignment shapes the next day · illustrative duties

The solver has to weigh this for 1,000 crew over 14 days at once: every yes on one duty narrows the options for later ones.

Rule	What it requires	Why it is hard for a solver
Eligibility	Only listed crew can take a duty	Limits choices; straightforward
Location continuity	Next duty starts where the last one ended; movement only by duty or permitted positioning	Links every duty to the one before it
Rest	11 h between duties; 23 h on transition days	Which rest applies depends on the day
Crew type and swing	Relief: 14 days on, 14 off. Excess: 6 on, 6 off	A pattern across many days
Shift start times	Relief times may change if other rules hold; Excess only lift-up or layback (±1 h)	A choice among start times per duty
Shift transitions	Day-to-night or night-to-day, with transition-day rest	Depends on the previous duties
CO duties	Crew-on rules and CO location restrictions	Applies only to certain duties
Return to base	On the last swing day or a transition day	Depends on the day's position in the swing

Rule	What it requires	Why it is hard for a solver
Existing roster	Commitments before, within and after the horizon	Fixes parts of each crew's sequence

The objective is to maximise covered duties; activity priorities are still to be confirmed. Several rules in the briefing need confirmation with operations before they are encoded.

Nearly every rule above has the shape "if X, then Y". That shape drives the solver choice in the rest of this doc.

How a MIP solver like Gurobi works

A MIP solver first solves an easier version where variables may take fractional values, then repairs the fractions. Its speed depends on how close that easier version is to the real problem.

Worked example

One crew member, **A**, starts at Howrah and is eligible for three duties:

Duty	Start	Route
D1	1 Jan 05:00	HWH → YAJ
D2	1 Jan 17:00	HWH → YAJ
D3	2 Jan 05:00	HWH → YAJ

Every pair is incompatible. D1 and D2 fall on the same day. After D1 or D2, A is in Yajpur and cannot start D3 at Howrah. The true answer is that A covers **one** duty.

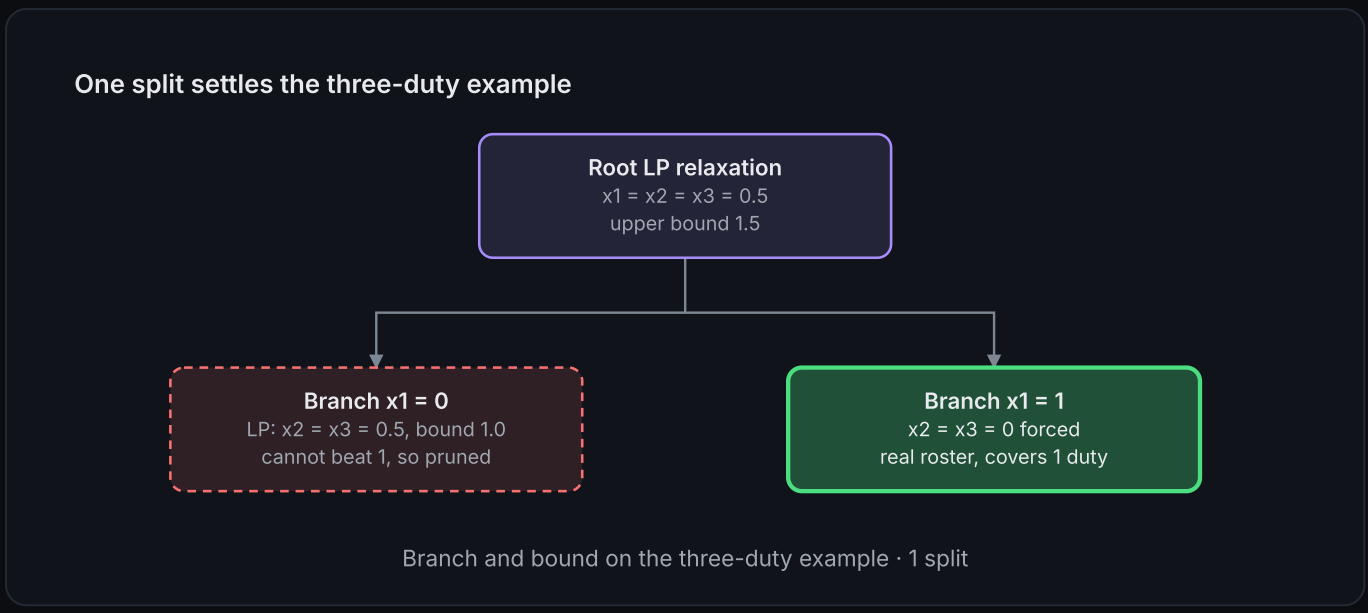
Step 1 – write linear constraints. With x1, x2, x3 ∈ {0, 1}:

$$\text{maximise } x_1 + x_2 + x_3 \quad \text{subject to} \quad x_1 + x_2 \le 1, \quad x_1 + x_3 \le 1, \quad x_2 + x_3 \le 1$$

Step 2 – solve the LP relaxation. Gurobi lets each x take any value from 0 to 1 and solves with the simplex method. The LP answer is x1 = x2 = x3 = 0.5, total 1.5. Physically this means "half of A on each duty", which is impossible, but 1.5 is a valid **upper bound**.

Step 3 – add cutting planes. Gurobi detects that all three duties conflict with each other and adds the clique cut x1 + x2 + x3 ≤ 1. The bound falls from 1.5 to 1. Our daily-cap constraint is already a clique constraint of this kind.

Step 4 – branch and bound. If the LP is still fractional, Gurobi picks a fractional variable and splits into two subproblems (x = 0 and x = 1). A branch whose LP bound cannot beat the best roster found so far is pruned.



Without the clique cut, Gurobi needs this split; with the cut, the root bound is already 1 and no branching is needed.

Step 5 – prove optimality. Gurobi tracks the **incumbent** (best real roster) and the **bound** (best still possible). When they meet, the roster is optimal. At our scale a log line might read: covered 283, bound 285.4, gap 0.8%.

Throughout, primal heuristics (rounding, fixing part of the problem and solving the rest) find good rosters early.

Why branch and bound slows down on our problem

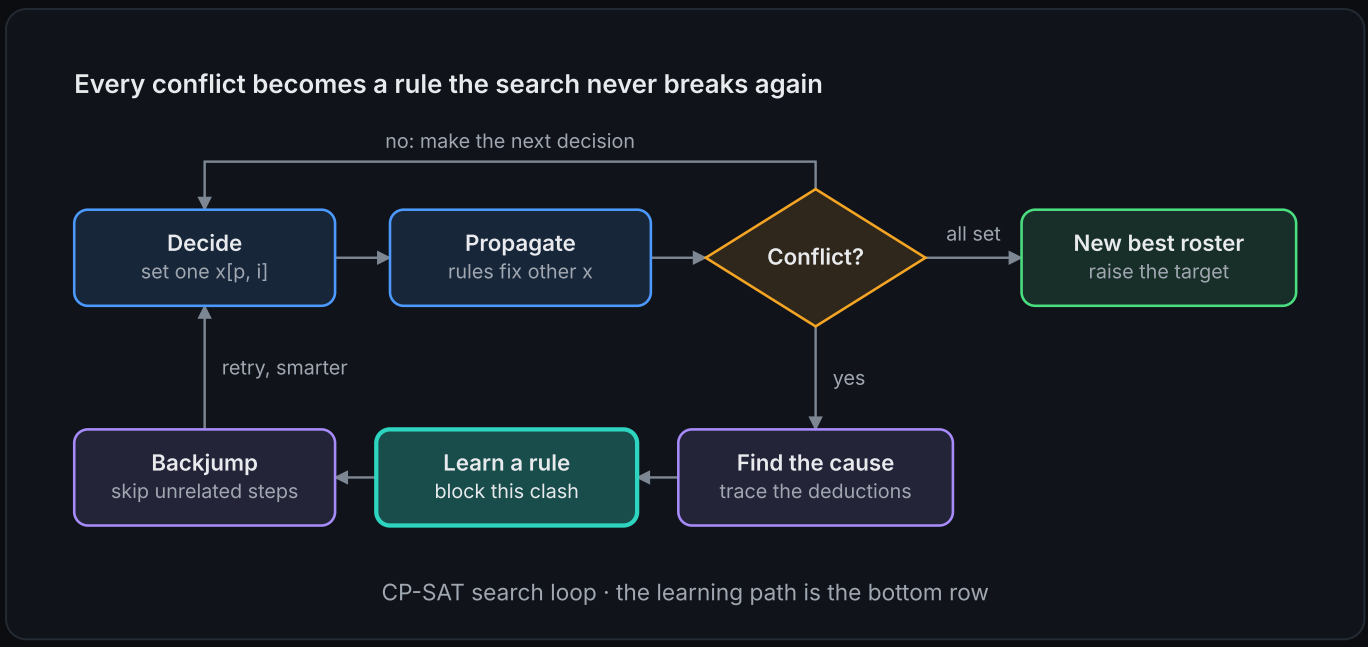
Branch and bound prunes a branch only when its LP bound cannot beat the best roster so far. On our full rule set this has two weaknesses:

- **Weak bounds.** The big-M constraints our if-then rules need (next section) let the LP "half-apply" rules, so bounds stay loose and few branches are pruned.
- **Little memory of why a branch failed.** When a branch is infeasible because, say, two crew assignments block a must-cover duty, branch and bound discards the branch. Modern MIP solvers do some conflict analysis, but it is a secondary feature. The same clash can be rediscovered in other parts of the tree.

CP-SAT is built the other way round: learning from each failure is its core mechanism.

How CP-SAT works: it learns from every dead end

CP-SAT never allows fractions. It makes true/false decisions, deduces their consequences, and learns a new rule from every dead end, so the search gets sharper the longer it runs.



The bottom row is what branch and bound lacks as its main mechanism: each failure leaves behind a rule that makes every later step smarter.

Step 1 – decide. In the three-duty example, CP-SAT tries x1 = true (A takes D1).

Step 2 – propagate. Constraints react at once: the D1–D2 conflict sets x2 = false, and the D1–D3 conflict sets x3 = false. One decision gives a complete roster covering one duty, with no fractional values anywhere.

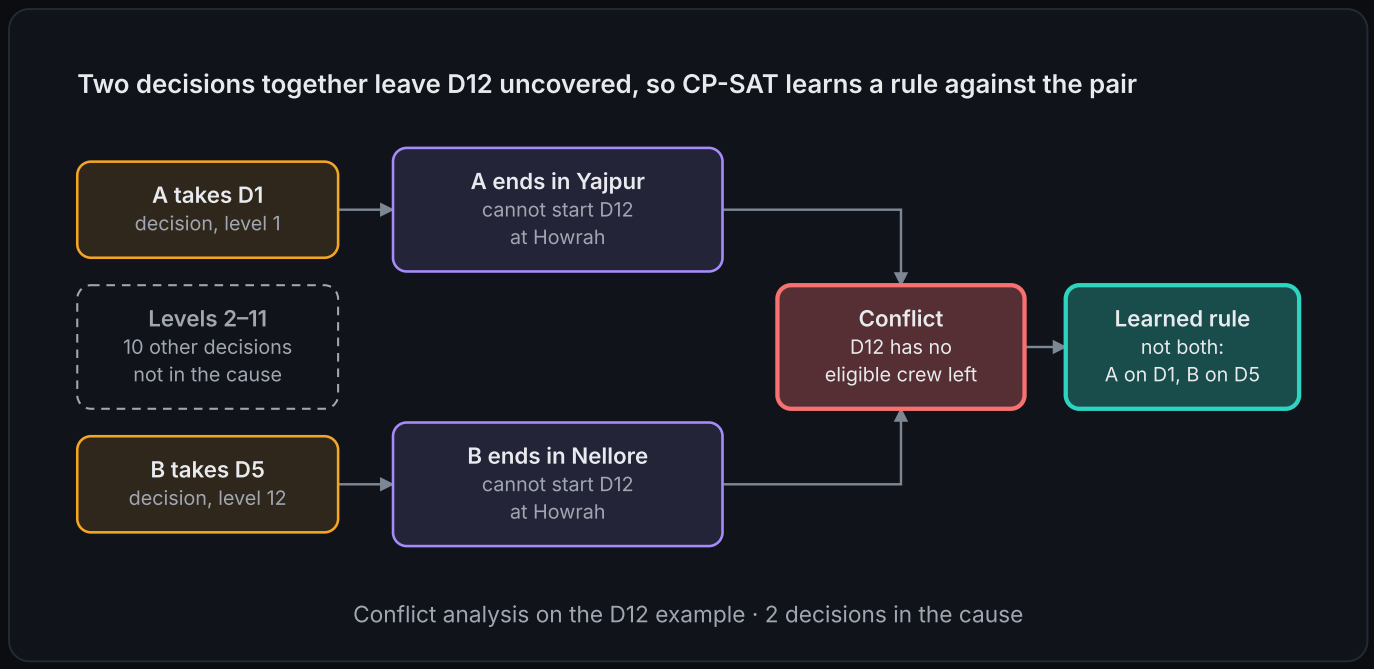
On the real problem, one decision such as "CREW_001 takes DUTY_001 (Howrah → Yajpur, 05:00, 1 Jan)" immediately:

- removes every other crew member from DUTY_001,
- limits CREW_001's next duty to one starting at Yajpur after the required rest,
- updates CREW_001's swing-day count and the day by which they must be back at base.

Step 3 – learn from conflicts (the key step). Suppose the search has made these decisions:

- Level 1: CREW_A takes D1 (Howrah → Yajpur).
- Levels 2–11: ten unrelated decisions elsewhere in the roster.
- Level 12: CREW_B takes D5 (Howrah → Nellore).

Propagation then finds that D12, which only A and B are eligible for, can no longer be covered, while covering it is required (a must-cover duty, or needed to beat the best roster so far). That is a conflict. CP-SAT traces it back to its cause.

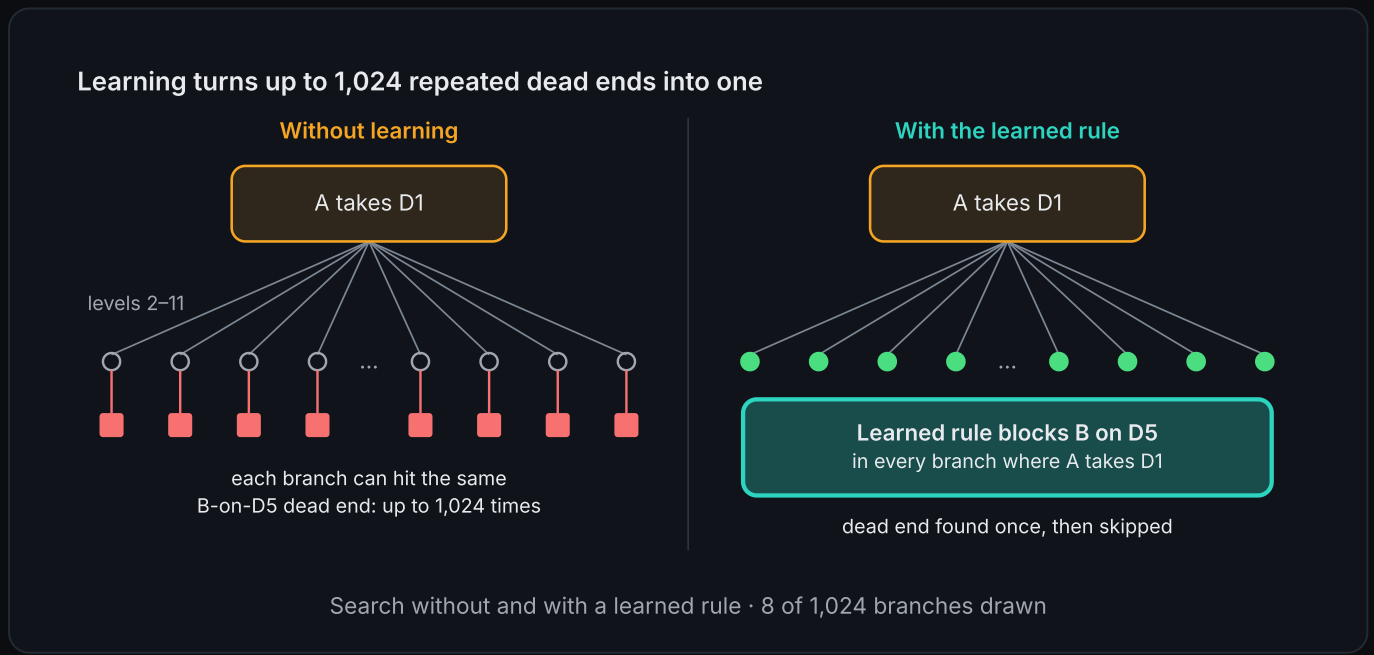


Conflict analysis on the D12 example · 2 decisions in the cause

The ten decisions at levels 2–11 played no part in the conflict. CP-SAT therefore **backjumps** straight to level 1 and keeps the learned rule for the rest of the run.

Why learning saves runtime

Without learning, a search that hits this conflict undoes only the last decision. The ten unrelated decisions have 2^10 = 1,024 combinations, and in each one the search can walk into the same A-on-D1-plus-B-on-D5 dead end again. With the learned rule, the dead end is found once and blocked in all 1,024 combinations.



Search without and with a learned rule · 8 of 1,024 branches drawn

Learned rules accumulate over a run, often into many thousands, and each one permanently removes a slice of the search. Crew conflicts repeat a lot (the same depots, shift times and eligibility pools), so one learned rule pays off many times. The 1,024 figure illustrates the mechanism; it is not a measured speed-up on our data.

Step 4 – bound and improve. CP-SAT also builds an LP relaxation internally for an upper bound on coverage, so it can prove optimality. In parallel, several workers run different strategies, including large neighbourhood search: keep most of the current roster, free one day or one depot, and re-optimise it. Learned rules stay in place across these restarts.

CP-SAT is therefore a hybrid, but the LP is a helper. The core engine is propagation plus conflict learning.

The big-M problem: why conditional rules hurt MIP

MIP can only write linear inequalities, so an if-then rule needs a large constant M to switch it on or off. This weakens the LP relaxation that MIP depends on.

Take the rest rule: if A does duty i and then duty j, j must start at least 11 h after i ends. With y = 1 meaning "A does i then j", and M = 336 h (the 14-day horizon):

$$\text{start}_j \;\geq\; \text{end}_i + 11 - M \cdot (1 - y_{ij})$$

When y = 1 the rule is enforced; when y = 0 it is switched off. In the LP relaxation, though, the solver can set y = 0.03. The rule is then almost switched off while the solver still gets most of the benefit of the sequence. The bound becomes loose, branching grows, and solving slows down.

In CP-SAT the same rule is written as an enforced constraint, with no M:

```
model.add(start[j] >= end[i] + 11).only_enforce_if(y[i, j])
```

The rule is active when y is true and ignored when y is false. There is no in-between value to exploit.

Most of our operational rules are of this kind:

Rule	MIP formulation	CP-SAT formulation
23 h rest only on transition days	Big-M on a transition indicator	`only_enforce_if(is_transition)`
Lift-up / layback (±1 h)	Integer time + big-M per option	`add_allowed_assignments` on start time
Swing pattern (14/14, 6/6)	Many linking constraints per day	Table or automaton constraint over the day sequence
Return to base on last swing day	Big-M on location per day	Enforced equality on last-day location
CO location restrictions	Big-M or many pairwise constraints	Enforced constraints or allowed tuples

Each MIP version is possible, but each one makes the model harder to read and its LP bound weaker.

Side-by-side comparison

Aspect	MIP (Gurobi)	CP-SAT
Core idea	Relax to fractions, then repair	Decide true/false, then deduce
Main engine	LP (simplex) + cutting planes + branch and bound	Propagation + conflict learning, with an LP helper
Remembers why a branch failed	Partly; conflict analysis is secondary, pruning is mainly by LP bound	Yes; every conflict becomes a rule used for the rest of the run
After a dead end	Discards the branch; the same clash can recur elsewhere	Backjumps past unrelated decisions; the clash is blocked everywhere
Variables	Continuous and integer	Boolean and bounded integer only
Proves optimality with	LP bound	LP bound plus exhausting the search
If-then rules	Big-M, which weakens the bound	Native `only_enforce_if`
Global constraints (circuit, tables, intervals)	Must be expanded into linear rows	Built in
Strongest when	LP relaxation is tight: flows, set partitioning	Many interacting logical rules
Our full rule set	Possible, but needs many big-M rows and careful tuning	Natural to model and extend
Good feasible solutions early	Yes, via heuristics	Yes, via parallel workers and LNS
Column generation / branch-and-price	Standard practice	Not built in
Licence	Commercial (free academic licence)	Free, open source (Apache 2.0)
Setup in Colab	Licence setup needed	`pip install ortools`

The free MIP solvers (HiGHS, CBC, SCIP) follow the same approach as Gurobi but are generally weaker on hard combinatorial models.

Model size at our scale

The full model is roughly 200,000 Boolean variables, within what CP-SAT handles routinely. The estimate uses the scoping figures and should be recomputed from real data.

Quantity	Calculation	Approximate count
Assignment variables x[p, i]	300 duties × 50 eligible crew	15,000

Quantity	Calculation	Approximate count
Eligible duties per crew (average)	15,000 ÷ 1,000 crew	15
Duty-to-duty arcs per crew	At most 15 × 14 ÷ 2 (arcs only go forward in time)	≤ 105
Start, end and self-loop arcs per crew	3 × 15	45
Sequencing literals	1,000 crew × about 150	about 150,000
Crew-day indicators (working, transition day, shift start, at base)	1,000 crew × 14 days × a few flags	tens of thousands

The main risk is skewed eligibility. Arcs grow with the square of a crew member's eligible duties, so one crew eligible for 100 duties alone adds up to about 5,000 arcs. We should check the eligibility distribution in the real data before scaling up.

Where MIP still wins, and honest caveats

CP-SAT is the right starting point, not a permanent answer. Five caveats matter:

1. **Full-scale rostering often ends in column generation.** Airlines and railways typically generate legal crew schedules with a shortest-path subproblem, then choose among them with a set-partitioning MIP (branch-and-price). That LP is very tight, which is where Gurobi is strongest. If CP-SAT stops scaling, this is the migration path, and rules tested in CP-SAT carry over as the legality checks for generated schedules.
2. **MIP solvers are not memoryless.** Gurobi and SCIP include conflict analysis too. The difference is emphasis: in MIP it supplements LP-bound pruning, while in CP-SAT it is the main engine.
3. **The core assignment structure suits MIP.** Without the if-then rules, each crew's duties form a path through a time-ordered graph, a flow structure with tight LP bounds. MIP is a useful cross-check on that core.
4. **A greedy heuristic finds a first roster faster.** Assigning duties in time order to any rested, correctly located, eligible crew takes milliseconds, but gives no quality guarantee. We will use it as a baseline and as a starting hint for CP-SAT.
5. **Metaheuristics such as Tabu Search give no bound.** They can help at large scale, but cannot say how far a roster is from optimal.

We have not benchmarked CP-SAT against Gurobi on our data. The comparison above reflects how the methods work, not measured results.

Recommendation and validation plan

Use CP-SAT as the primary solver, seeded with a greedy roster, and cross-check it against a MIP on small instances.

- [ ] Build tiny hand-checkable instances (3–10 duties) with known optimal coverage, including the three-duty example above.
- [ ] Write a greedy baseline and pass its roster to CP-SAT with `model.add_hint(...)`.
- [ ] Run CP-SAT with `max_time_in_seconds`, `num_workers=8` and `log_search_progress=True`.
- [ ] Record status, covered duties, bound, gap and runtime for every run.
- [ ] Write the core sequencing model as a flow MIP in HiGHS (free) or Gurobi (academic licence) and confirm both solvers report the same optimum.
- [ ] Check the eligibility distribution in real data before scaling to 1,000 crew.
- [ ] Benchmark both solvers on the same instances as each operational rule is added.
- [ ] Revisit column generation if CP-SAT runtime becomes unacceptable at full scale.